EngSci 721

Inverse Problems and Learning From Data

Oliver Maclaren (oliver.maclaren@auckland.ac.nz)

Lecture 13: 'Cheap' Physics-Informed Machine Learning

Topics:

- Hybrid statistical/machine learning and physics-informed modelling
- Basis function expansions and splines
- Differential equations as regularisation

Neural networks? Next lecture?

Module overview

Inverse Problems and Learning From Data (*Oliver Maclaren*)
[~14 lectures/3 tutorials]

1. *Basic concepts* [5 lectures + 1 Tutorial]

Forward vs inverse problems. Well-posed vs ill-posed problems. Algebra and calculus of inverse problems (left and right inverses, generalised and pseudo inverses, resolution operators, matrix calculus). Representing higher dimensional problems (image data etc).

2. *Instability and regularisation* [6 lectures + 1 Tutorial]

Instability and related issues for generalised inverses. Introduction to regularisation and trade-offs. Tikhonov regularisation. Higher-order Tikhonov regularisation. Sparsity and regularisation using different norms. Truncated singular value decomposition. Iterative regularisation, including stochastic/mini-batch gradient descent.

2. *Further topics* [3 lectures + 1 Tutorial]

Regularisation parameter choice, including statistical and machine learning views of regularisation. Confidence sets for linear and nonlinear models. Physics-informed machine learning and neural networks.

Eng Sci 721 : Lecture 13

'Cheap' physics-informed machine/stat. learning

'Hybrid' statistical/ML/data & physical models } 'scientific/eng. machine learning'

- Splines (data model)
  +
- Differential equations as penalty ('physical' model as regularisation)

Neural nets? Next lecture! (?)

→

Basic idea

We can make the 'data fitting' model class more flexible

+

make the 'regularisation' more sophisticated/scientific

eg

$$\|y - f(x)\|^2 + \lambda \|M(x)\|^2$$

flexible stat./ML model like neural network, spline, gaussian process etc.

scientific model eg differential eq^n etc

let's consider a time dependent problem & ODEs

Basis functions

Here we will take a simple approach to constructing flexible data fitting models (see next lecture for neural networks)

Thus, recall linear forward mappings can represent seemingly nonlinear functions as long as we make it linear in parameters

This leads to eg 'basis function expansions', ie 'linear combinations of features'.

For example, a time-dependent function can be written

$$y(t) \approx \sum_{j=1}^{n} \beta_j b_j(t)$$

for basis functions $b_j(t)$ & coefficients (parameters) β_j.

Functions, discretisation levels

$$y(t) = \sum_j b_j(t) \beta_j$$

represents a time-dep. function as a linear combination of other time-dep. functions

→ we can think of functions as infinite-dimensional vectors

↳ eg add pointwise, multiply by scalars etc.

→ more practically, we can introduce an 'underlying' fine but discrete grid we evaluate our function on to represent it as a large vector

↳ this is distinct from the grid we observe the function on:

fine vector → F → observed vector

eg our unknown can be a function / large vector

Discretised basis function expansion

Using a fine time grid $t_1, \dots, t_d$ to convert our infinite dimensional / continuous problem to a large but finite dim. discrete prob., we get

$$\begin{bmatrix} y(t_1) \\ y(t_2) \\ \vdots \\ y(t_d) \end{bmatrix} = \begin{bmatrix} b_1(t_1) & b_2(t_1) & \dots & b_n(t_1) \\ b_1(t_2) & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \\ b_1(t_d) & b_2(t_d) & & b_n(t_d) \end{bmatrix} \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_n \end{bmatrix}$$

ie

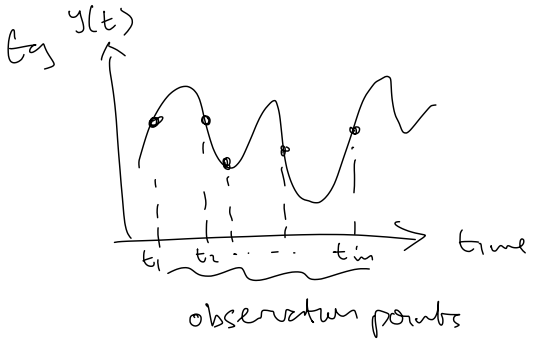
$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_d \end{bmatrix} = \begin{bmatrix} b_{11} & \cdots & b_{1n} \\ b_{21} & & \vdots \\ \vdots & & \vdots \\ b_{d1} & & b_{dn} \end{bmatrix} \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_n \end{bmatrix}$$

or

$$y = B\beta \qquad \text{for vectors } y, \beta \text{ matrix } B.$$

Underlying grid vs observed grid

In dynamic physical systems we are often interested in the 'underlying' continuous, or at least fine-scale, function, but only have a much coarser, finite, discrete set of observations



We can often represent this (in principle) by a linear observation matrix/operator:

$$y_{obs} = A_{obs}\, y_{fine}$$

→

Observation matrices

The standard basis vectors are:

$$e_j = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \leftarrow j^{th} \text{ entry}$$

& $e_j^T A = \text{row}_j(A)$

$$Ae_j = \text{col}_j(A)$$

Since in our inverse problem setting each row of the system corresponds to an obs. the relation

$$y_{obs} = A_{obs} I_{fine}$$

can be realised by building A_{obs} from rows e_j^T for any fine grid point j that has an obs

eg

$$A_{obs} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & \dots \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \dots \end{bmatrix} \begin{matrix} \} \text{ obs } 5^{th} \\ \} \& \, 7^{th} \\ \text{grid points} \end{matrix}$$

Splines & B Splines

One way to represent smooth functions of time is with splines

→ these are piecewise polynomials defined by the degree of the polynomials & the properties at the 'join' points, called knots

Splines can be represented by basis functions — B-splines

↳ any degree d polynomial spline (d: highest power of piecewise polys) can be rep. by a linear combo of B splines of this degree

Splines are defined by degree & location of internal join points, called knots

At non-repeated knots, spline has $d-1$ continuous derivatives

↳ eg piecewise linear: $d=1$

↳ first derivative not continuous, value is

Working with splines

Many libraries for constructing spline bases take two key param:

- polynomial degree d
- 'degrees of freedom' ν = number of basis functions

→ number of knots = $\nu - d$

Libraries allow us to construct 'design' / 'feature' matrices a la our 'finely discretised' functions

→ often automatically add an 'all ones' (constant feature) column

→ add with '1 + basisfunc.'

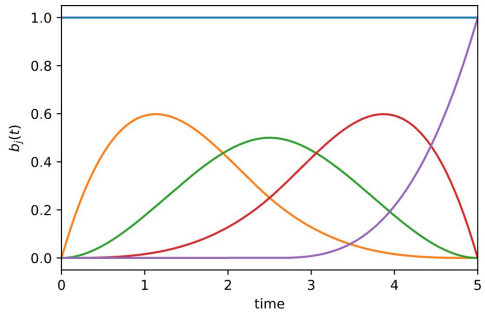
→ remove with 'basisfunc-1'

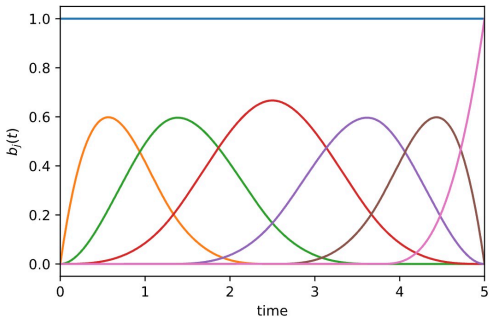
→

Example

We will use 'Patsy' in Python here.

```
import numpy as np
import matplotlib.pyplot as plt
from patsy import dmatrices, dmatrix

# (fine) time grid
t = np.linspace(0., 5., 1000)

# Four cubic basis functions, number of implied knots = 4-3 = 1,
# as well as one constant function.
# Number of columns of X is 4 + 1 = 5.

B = dmatrix("1 + bs(t, df=4, degree=3)", {"t": t})
plt.plot(t,np.asarray(B))
plt.xlabel(r'time')
plt.xlim(0,5)
plt.ylabel(r'$b_j(t)$')
plt.show()

# Six cubic basis functions, number of implied knots = 6-3 = 3,
# as well as one constant function.
# Number of columns of X is 6 + 1 = 7.
B = dmatrix("1+bs(t, df=6, degree=3)", {"t": t})
plt.plot(t,np.asarray(B))
plt.xlabel(r'time')
plt.xlim(0,5)
plt.ylabel(r'$b_j(t)$')
plt.show()
```





Fitting splines

we can fit splines in the way we any linear model → least squares!

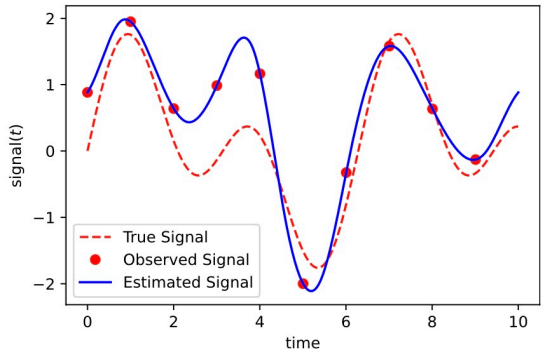
make sure to use Aobs for relating to data, but plot without

Example:

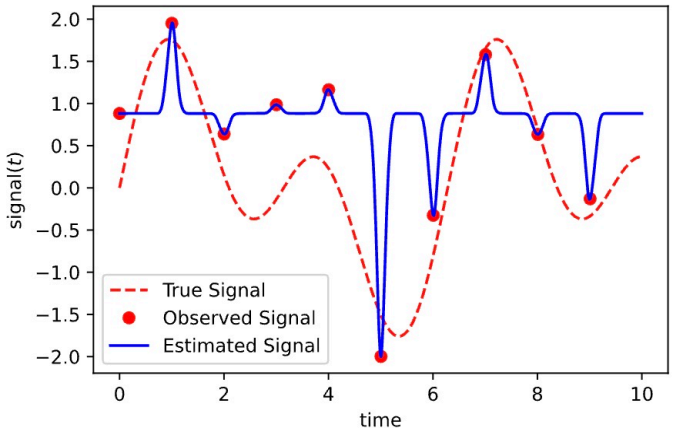
keep obs vars →

basis →

```
# generate some data to fit to
t = np.linspace(0., 10., 1000)
fine_signal = np.sin(t) + np.sin(2*t)
obs_indices = np.arange(0, 1000, 100)
A_obs = np.eye(len(fine_signal))[obs_indices]
np.random.seed(0)
obs_signal = A_obs @ fine_signal + np.random.normal(scale=0.5, size=A_obs.shape[0])
# fit
B = dmatrix("1 + bs(t, df=15, degree=3)", {"t": t})
B_obs = A_obs@B
beta_est = np.linalg.pinv(B_obs)@obs_signal #should really use lstsq

plt.plot(t, fine_signal, 'r--',label='True Signal')
plt.plot(A_obs@t, obs_signal, 'ro', label='Observed Signal')
plt.plot(t, B@beta_est, 'b', label='Estimated Signal')
plt.xlabel(r'time')
plt.ylabel(r'$\text{signal}(t)$')
plt.legend()
plt.show()
```



More spline basis functions



simpler?

→ related to double descent?

Regularisation

We have previously seen the use of derivative matrices for regularisation, eg higher order Tikhonov

Recall: $D_1 = \frac{1}{\Delta t}\begin{bmatrix} -1 & 1 & 0 & \dots \\ 0 & -1 & 1 & 0 \dots \\ & \dots & 0 & -1 & 1 \end{bmatrix}$

[here we include the explicit Δt for scaling, but this can also be absorbed into λ later]

we can also use eg (using zero indexing) } — Python

$D_2 = D_1[1:,1:]@D$

$D_3 = D_1[2:,2:]@D[1:,1:]@D$

ie drop a row & col each time, giving differential operators defined just on 'internal' points, no BC

Example

```
# construct first derivative matrix
n = len(t)
D = (np.diag(-np.ones(n)) + np.diag(np.ones(n-1),1))[:-1,:]
print('D')
print(D)
print('D2')
print(D[1:,1:]@D)
print('D3')
print(D[2:,2:]@D[1:,1:]@D)
```

gives:

```
D
[[-1.  1.  0. ...  0.  0.  0.]
 [ 0. -1.  1. ...  0.  0.  0.]
 [ 0.  0. -1. ...  0.  0.  0.]
 ...
 [ 0.  0.  0. ...  1.  0.  0.]
 [ 0.  0.  0. ... -1.  1.  0.]
 [ 0.  0.  0. ...  0. -1.  1.]]
D2
[[ 1. -2.  1. ...  0.  0.  0.]
 [ 0.  1. -2. ...  0.  0.  0.]
 [ 0.  0.  1. ...  0.  0.  0.]
 ...
 [ 0.  0.  0. ...  1.  0.  0.]
 [ 0.  0.  0. ... -2.  1.  0.]
 [ 0.  0.  0. ...  1. -2.  1.]]
D3
[[-1.  3. -3. ...  0.  0.  0.]
 [ 0. -1.  3. ...  0.  0.  0.]
 [ 0.  0. -1. ...  0.  0.  0.]
 ...
 [ 0.  0.  0. ...  1.  0.  0.]
 [ 0.  0.  0. ... -3.  1.  0.]
 [ 0.  0.  0. ...  3. -3.  1.]]
```

Differential equations as regularisation

when we regularise by derivatives like

$$\|D_1 x\|^2 \text{ or } \|D_2 x\|^2$$

etc, we are effectively approximately enforcing a differential equation! Eg $D_1 x \approx 0$

- when applied to the parameters this looks like eg $\|D_1 \theta\|^2$ etc

- However, in the context of basis functions & physical modelling it often makes sense to apply to the differential equation governing the true scale.

Eg $\|D_1 B\beta\|^2$ not $\|D_1 \beta\|^2$ ← coeff.

↓

ie $D_1 B\beta = \boxed{D_1 y \approx 0}$

↑ actual smooth function (recall also deconvolution)

Example.

consider simple projectile motion & vertical height y

$$\Rightarrow \frac{d^2y}{dt^2} = -g \quad , \quad \begin{matrix} y(0) = y_0 \\ y'(0) = v \end{matrix}$$

internal domain — 'boundary' (initial) cond.

let: $y = B\beta$,

then: $\boxed{D_2 B\beta = -g \cdot 1}$ (discrete eqn at internal points)

where: $y \in \mathbb{R}^d$, $B \in \mathbb{R}^{d \times n}$, ← note

$D_2 \in \mathbb{R}^{(d-2) \times d}$ ↑ note, $1 \in \mathbb{R}^{d-2}$

&

$D_2 = D_1[1:, 1:]@D$

$$= \frac{1}{\Delta t^2} \begin{bmatrix} 1 & -2 & 1 & 0 & \cdots \\ 0 & 1 & -2 & 1 & \cdots \\ \vdots & & \ddots & & \\ 0 & 0 & \cdots & 1 & -2 & 1 \end{bmatrix}$$

Combine with observations:

instead of (tho' can use these too) explicit IC, use data!

$$y_{obs} \approx A_{obs} y_{fine} = A_{obs} B\beta$$

$$D_2 y_{fine} = D_2 B\beta \approx -1\, g$$

$$\Rightarrow \| y_{obs} - A_{obs} B\beta \|^2 + \lambda \| D_2 B\beta - (-g)1 \|^2$$

$$\Rightarrow \left\| \begin{bmatrix} A_{obs} B \\ D_2 B \end{bmatrix} \beta - \begin{bmatrix} y_{obs} \\ (-g)1 \end{bmatrix} \right\|^2$$

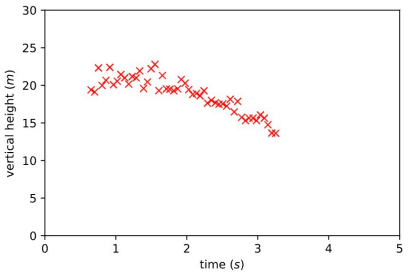
ie $\| \tilde{A}\beta - \tilde{y} \|^2$

& $\beta = \tilde{A}^+ \tilde{y}$

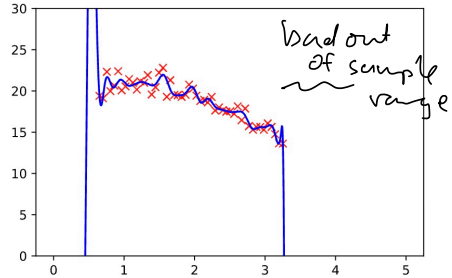
Example (challenge: do! past assignment)

- Projectile motion with air resistance + gravity
- Fit spline + gravity only reg.
  - ↳ so trade-off

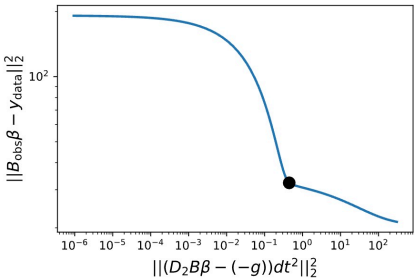
data:



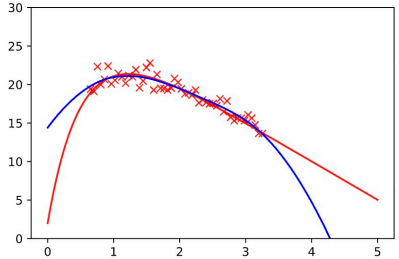
spline only (50 dof)



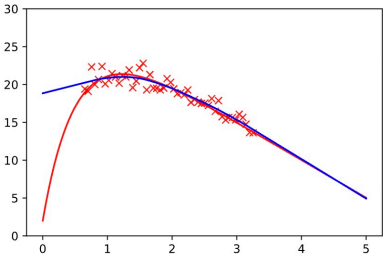
L-curve



gravity only



air resistance only:



Misspecification, inductive bias, regularisation

If the model is correct there should be no trade-off (in principle) in enforcing more & more strongly

→ contrast to 'simple' regularisation like $D_1 x \approx 0$

If the model is incorrect, then a trade-off should appear

ML folk like to call the 'biases' we put in beyond the data, 'inductive bias'

↳ unavoidable (see eg Hume!)

→

Linear vs nonlinear?

We can easily apply the approach so far to linear ODEs with known parameters

→ If we want unknown parameters or nonlinear ODEs, then can do eg 'successive approx' or 'iteration' (fixed point; successive)

$$DB\beta = \alpha B\beta$$

↳ nonlinear if α unknown.

Simple approach: take current guess of α, solve for β, update α.

Alternative: direct numerical optimisation

Higher dimensions?

- Splines can be extended but other approaches eg neural nets typically more pop. in high dimension probs.

→ next time.